

Operating Systems Study Guide

PART 2 & 3: PROCESS MANAGEMENT, THREADS & CPU SCHEDULING

1. Explain Process State Transition with Neat Diagram.

A process is a program in execution. During its lifetime, a process moves through different states. The movement of a process from one state to another is called process state transition.

Process States

New State: When a process is created, it enters the New state. The operating system allocates necessary resources and initializes the process.

Ready State: After creation, the process enters the Ready state. It is ready to execute and waits in the ready queue for CPU allocation.

Running State: When the CPU scheduler assigns the processor, the process enters the Running state and executes its instructions.

Waiting (Blocked) State: If the process requests an I/O operation or waits for an event, it moves to the Waiting state until the event occurs.

Terminated State: After completing execution, the process enters the Terminated state and releases all allocated resources.

[Diagram: Process State Transition (New → Ready → Running → Waiting/Terminated)]

State Transitions:

- **New → Ready** : Process is admitted into memory.
- **Ready → Running** : CPU is allocated by the scheduler.
- **Running → Waiting** : Process requests I/O or waits for an event.
- **Waiting → Ready** : Event completion or I/O completion.
- **Running → Terminated** : Process finishes execution.

2. What is PCB? Explain with Diagram.

A Process Control Block (PCB) is a data structure maintained by the operating system for each process. It contains all the information required to manage and execute a process.

Purpose of PCB: The PCB stores process-related information and helps the operating system perform scheduling and context switching efficiently.

Process State: Stores the current state of the process such as New, Ready, Running, Waiting, or Terminated.

Process ID (PID): Each process is assigned a unique identifier known as Process ID. It is used to identify and manage processes.

Program Counter: Stores the address of the next instruction to be executed when the process resumes execution.

CPU Registers: Contains the values of CPU registers that must be saved during context switching.

Memory Management Information: Stores details about memory allocation such as page tables, segment tables, and memory limits.

I/O Status Information: Contains information about files opened, devices allocated, and pending I/O operations.

Accounting Information: Stores CPU usage, execution time, priority, and other process statistics.

[Diagram: PCB Structure Block (State, PID, PC, Registers, Memory, I/O)]

3. Explain Process Scheduling.

Process scheduling is the activity of selecting a process from the ready queue and allocating the CPU to it. It enables efficient use of the processor and improves overall system performance.

Need for Process Scheduling: In a multiprogramming environment, many processes compete for CPU time. Process scheduling determines which process should execute next.

Ready Queue: All processes that are ready to execute are placed in the ready queue. The scheduler selects one process from this queue.

CPU Scheduler: The CPU scheduler chooses a process from the ready queue and assigns the processor to it for execution.

Scheduling Objectives:

- **CPU Utilization:** The scheduler aims to keep the CPU busy as much as possible.
- **Throughput:** It increases the number of processes completed per unit time.
- **Turnaround Time:** It attempts to reduce the total time required to complete a process.
- **Waiting Time:** It minimizes the time a process spends waiting in the ready queue.
- **Response Time:** It improves the responsiveness of interactive systems.

Scheduling Algorithms: Common scheduling algorithms include FCFS, SJF, Priority Scheduling, and Round Robin.

4. With Diagram Explain Queuing Diagram Representation of Process Scheduling.

The queuing diagram represents the movement of processes among different queues and resources in a system. It helps visualize process scheduling and execution.

Job Queue: All newly created processes enter the job queue. These processes are waiting to be admitted into main memory.

Ready Queue: Processes loaded into memory and ready for execution are placed in the ready queue.

CPU: The CPU scheduler selects a process from the ready queue and allocates the processor.

Device Queue: Processes waiting for I/O operations are placed in device queues.

[Diagram: Queuing Model Representation (Job Pool → Ready Queue → CPU → Device Queue)]

Working: A process enters the job queue when created. It is then admitted to the ready queue. After CPU allocation, it executes. If I/O is required, the process moves to a device queue. After I/O completion, it returns to the ready queue. When execution finishes, the process exits the system.

5. Write a Note on Schedulers.

A scheduler is an operating system component that selects processes for execution and controls their movement between queues.

Long-Term Scheduler: The long-term scheduler selects processes from the job pool and loads them into main memory for execution. Its main function is to control the degree of multiprogramming and maintain a balance between CPU-bound and I/O-bound processes.

Short-Term Scheduler: The short-term scheduler selects a process from the ready queue and allocates the CPU. It executes very frequently and therefore must be fast and efficient.

Medium-Term Scheduler: The medium-term scheduler temporarily removes processes from memory and stores them on secondary storage. This process, called swapping, helps reduce memory load and improves system performance.

Importance of Schedulers: Schedulers improve CPU utilization, reduce waiting time, and ensure efficient execution of processes.

6. Write a Note on Cooperating Processes.

A cooperating process is a process that can affect or be affected by the execution of another process. Such processes work together and share information to accomplish common tasks.

Information Sharing: Several processes may need access to common data. Cooperating processes allow efficient sharing of information among users and applications.

Computation Speedup: Large tasks can be divided into smaller subtasks and executed by multiple cooperating processes, reducing overall execution time.

Modularity: Complex systems can be divided into smaller modules that operate as separate cooperating processes. This improves system design and maintenance.

Convenience: Users can perform multiple activities simultaneously, such as editing a document while printing another file.

Interprocess Communication (IPC): Cooperating processes communicate through Shared Memory (processes share a common memory area to exchange data) or Message Passing (processes exchange information by sending and receiving messages). These communication mechanisms ensure coordination among cooperating processes.

7. Write a Note on Threads.

A thread is the smallest unit of CPU execution within a process. A process may contain multiple threads that execute concurrently while sharing the same resources.

Components of a Thread: Each thread consists of a Thread ID, Program Counter, Register Set, and Stack. Threads within the same process share code, data, and files.

Types of Threads: User-Level Threads (managed by user-level libraries rather than the OS kernel; faster thread operations) and Kernel-Level Threads (managed directly by the OS kernel; provide better concurrency across multiple processors).

Advantages of Threads: Responsiveness, Resource Sharing, Economy, and Parallelism on multiprocessor systems.

8. Explain the Benefits of Threads.

Responsiveness: A multithreaded application remains responsive even when one thread is blocked or performing a lengthy operation.

Resource Sharing: Threads belonging to the same process share code, data, and system resources. This makes communication between threads easier and faster than communication between separate processes.

Economy: Creating and managing threads requires less memory and overhead compared to processes. Context switching between threads is also faster.

Parallelism: On multiprocessor systems, multiple threads can execute simultaneously on different processors, increasing throughput.

Better CPU Utilization: Threads allow multiple activities to be performed concurrently, reducing CPU idle time.

9. Explain Preemptive and Non-Preemptive Scheduling.

CPU scheduling can be classified into Preemptive and Non-Preemptive Scheduling based on whether a running process can be interrupted.

Preemptive Scheduling: The operating system can interrupt a running process and allocate the CPU to another process. It features higher context switching overhead but better responsiveness (e.g., Round Robin, SRTF).

Non-Preemptive Scheduling: Once a process gets the CPU, it continues execution until it terminates or voluntarily releases the CPU. It is simple to implement but results in poorer response time for interactive systems (e.g., FCFS, Non-preemptive SJF).

Preemptive Scheduling	Non-Preemptive Scheduling
CPU can be interrupted	CPU cannot be interrupted
Better response time	Higher waiting time
More overhead	Less overhead
Suitable for interactive systems	Suitable for batch systems

10. List and Explain Various Scheduling Criteria.

CPU Utilization: Measures the percentage of time the CPU is busy executing processes. The objective is to maximize it.

Throughput: Refers to the number of processes completed per unit time. Higher throughput indicates better performance.

Turnaround Time: Total time taken from process submission to process completion (Turnaround Time = Completion Time – Arrival Time). The objective is to minimize it.

Waiting Time: Total time a process spends waiting in the ready queue before execution. The scheduler should minimize it.

Response Time: Time between submitting a request and receiving the first response, crucial in interactive systems.

Fairness: The scheduler should allocate CPU time fairly among all processes and prevent starvation.

11. Explain FCFS Scheduling with an Example.

First Come First Serve (FCFS) follows the FIFO principle where the process that arrives first gets the CPU first.

Example: Processes P1 (Burst=5), P2 (Burst=3), P3 (Burst=2).

Gantt Chart: | P1 (0-5) | P2 (5-8) | P3 (8-10) |

Waiting Time: P1 = 0, P2 = 5, P3 = 8.

Average Waiting Time: $(0 + 5 + 8) / 3 = 13 / 3 = 4.33$ ms

Advantages: Simple and easy to implement. **Disadvantages:** Suffers from the convoy effect.

12. Explain Shortest Job First (SJF) Scheduling with an Example.

SJF scheduling selects the process with the smallest CPU burst time for execution, minimizing average waiting time.

Example: Processes P1 (Burst=6), P2 (Burst=8), P3 (Burst=3), P4 (Burst=4).

Execution Order: P3 → P4 → P1 → P2

Gantt Chart: | P3 (0-3) | P4 (3-7) | P1 (7-13) | P2 (13-21) |

Waiting Time: P3 = 0, P4 = 3, P1 = 7, P2 = 13.

Average Waiting Time: $(0 + 3 + 7 + 13) / 4 = 23 / 4 = 5.75$ ms

13. Explain Round Robin Scheduling Algorithm with an Example.

Round Robin (RR) is a preemptive algorithm designed for time-sharing systems where each process gets a fixed time quantum.

Example: Time Quantum = 2 ms. Processes P1 (Burst=5), P2 (Burst=4), P3 (Burst=2).

Gantt Chart: | P1 (0-2) | P2 (2-4) | P3 (4-6) | P1 (6-8) | P2 (8-10) | P1 (10-11) |

Execution Iterations: P1 executes for 2 ms (rem 3); P2 executes for 2 ms (rem 2); P3 executes for 2 ms (completed); P1 executes for 2 ms (rem 1); P2 executes for 2 ms (completed); P1 executes for 1 ms (completed).

14. Explain Priority Scheduling with an Example.

Priority Scheduling allocates the CPU to the process with the highest priority (lower number indicates higher priority).

Example: P1 (Burst=10, Pri=3), P2 (Burst=1, Pri=1), P3 (Burst=2, Pri=4), P4 (Burst=1, Pri=2).

Execution Order: P2 → P4 → P1 → P3

Gantt Chart: | P2 (0-1) | P4 (1-2) | P1 (2-12) | P3 (12-14) |

Waiting Time: P2 = 0, P4 = 1, P1 = 2, P3 = 12.

Average Waiting Time: $(0 + 1 + 2 + 12) / 4 = 3.75$ ms

15. Explain Multilevel Queue Scheduling.

Multilevel Queue Scheduling divides the ready queue into multiple separate queues (e.g., System, Interactive, Batch) based on process characteristics. Processes are permanently assigned to a queue.

[Diagram: Multilevel Queue Hierarchy (Foreground RR Queue → Background FCFS Queue)]

Scheduling: Fixed priority scheduling or time-slicing is used between queues. Higher-priority queues are served before lower-priority queues. Within queues, individual algorithms apply (e.g., Foreground Queue uses Round Robin, Background Queue uses FCFS).

16. Explain Multilevel Feedback Queue Scheduling.

Multilevel Feedback Queue Scheduling allows processes to move between queues based on their behavior and CPU usage to optimize responsiveness and prevent starvation.

Working Principle: A process initially enters the highest-priority queue. If it uses too much CPU time, it is moved to a lower-priority queue.

Example Configuration:

Queue 0 → Round Robin (8 ms) [Processes move down if not completed]

Queue 1 → Round Robin (16 ms) [Processes move down if not completed]

Queue 2 → FCFS

17. Describe the Differences Between Short-Term, Medium-Term and Long-Term Schedulers.

Feature	Long-Term Scheduler	Medium-Term Scheduler	Short-Term Scheduler
Selects From	Job Queue	Memory Processes	Ready Queue
Purpose	Admit Processes	Swapping	CPU Allocation
Speed	Slow	Moderate	Fast
Frequency	Less Frequent	Occasional	Very Frequent

18. Explain Multiprocessor Scheduling.

Multiprocessor Scheduling manages systems with multiple CPUs. It is split into Asymmetric Multiprocessing (AMP) where one master processor makes scheduling decisions, and Symmetric Multiprocessing (SMP) where all processors are equal and schedule independently.

Processor Affinity: A process prefers to run on the same CPU where it executed previously (Soft Affinity attempts it; Hard Affinity enforces specific processors).

Load Balancing: Distributes processes evenly among all processors to avoid overloading an individual CPU.

19. Round Robin Scheduling (Time Quantum = 5 ms)

Given: P1 (Burst=6), P2 (Burst=8), P3 (Burst=7), P4 (Burst=3).

Gantt Chart: | P1 (0-5) | P2 (5-10) | P3 (10-15) | P4 (15-18) | P1 (18-19) | P2 (19-22) | P3 (22-24) |

Completion Times: P1 = 19, P2 = 22, P3 = 24, P4 = 18.

Waiting Time (Completion Time – Burst Time):

P1: $19 - 6 = 13$ | P2: $22 - 8 = 14$ | P3: $24 - 7 = 17$ | P4: $18 - 3 = 15$

Average Waiting Time: $(13 + 14 + 17 + 15) / 4 = 59 / 4 = 14.75$ ms

20. SJF Scheduling – Find Average Turnaround Time

Given: P1 (Burst=15), P2 (Burst=4), P3 (Burst=10), P4 (8), P5 (5).

Execution Order: P2 → P5 → P4 → P3 → P1

Gantt Chart: | P2 (0-4) | P5 (4-9) | P4 (9-17) | P3 (17-27) | P1 (27-42) |

Turnaround Time (Equal to Completion Time as Arrival Time = 0):

P2 = 4, P5 = 9, P4 = 17, P3 = 27, P1 = 42.

Average Turnaround Time: $(4 + 9 + 17 + 27 + 42) / 5 = 99 / 5 = 19.8$ ms

21(a). Draw Gantt Charts

FCFS Scheduling: (Order: P1 → P2 → P3 → P4 → P5)

| P1 (0-2) | P2 (2-3) | P3 (3-11) | P4 (11-15) | P5 (15-20) |

SJF Scheduling: (Burst Order: P2=1, P1=2, P4=4, P5=5, P3=8)

| P2 (0-1) | P1 (1-3) | P4 (3-7) | P5 (7-12) | P3 (12-20) |

Non-Preemptive Priority: (Priority Order: P3=4, P5=3, P1=2, P4=2, P2=1)

| P3 (0-8) | P5 (8-13) | P1 (13-15) | P4 (15-19) | P2 (19-20) |

Round Robin (Quantum = 2):

| P1 (0-2) | P2 (2-3) | P3 (3-5) | P4 (5-7) | P5 (7-9) | P3 (9-11)
| P4 (11-13) | P5 (13-15) | P3 (15-16) | P5 (16-18) | P3 (18-20) |

21(b). Turnaround Time for Each Algorithm

FCFS: P1 = 2, P2 = 3, P3 = 11, P4 = 15, P5 = 20

SJF: P2 = 1, P1 = 3, P4 = 7, P5 = 12, P3 = 20

Priority Scheduling: P3 = 8, P5 = 13, P1 = 15, P4 = 19, P2 = 20

Round Robin (q = 2): P1 = 2, P2 = 3, P4 = 13, P5 = 18, P3 = 20